Checking the Learned Dissipativity Conditions on the All-VSG Kundur Example

Sehaj Randhir Singh, a note for Dr. Andrey Gorbunov and Dr. Han Wang, October 2026. Simulation only. Everything here is my rebuild of the all-VSG Kundur example in Wang, Wang, Zhuang, Moffat and Dörfler, arXiv:2510.22324.

The short version

I wanted to know whether the learned conditions hold where the stability theorem actually uses them, along the closed loop at $u = \pi(x)$, and not only at the training tuples. In my rebuild they don't. On five seeds the dissipativity inequality holds at the training tuples, but along the closed loop it fails on 75% to 99% of the region X, and on 94% to 100% of the states near the equilibrium.

When I went looking for why, it came down to scale. Near the equilibrium the inequality asks V to drop by 2.8% to 24% of itself in one 0.5 ms step, while a step only delivers at most 0.007%, and the plant's own damping only allows about 0.03%. Φ was trained to be positive definite, but nothing ties how big it is to the sampling step, and in discrete time it has to be of order Δt.

Fixing that fixes most of it. When I made Φ positive definite by construction and trained the inequality at $u = \pi(x)$, the violation came down to 8% to 23% of X on all five seeds, with Φ positive definite everywhere. What's left sits in one place, at large angle and almost zero frequency deviation, and the paper's fault trajectory runs right through that band. That's the part I'd most like your read on.

\g<1>I couldn\'t find code for the example, so I rebuilt it from the paper, keeping its parameters wherever the paper gives them. The model is the Kundur two-area system with all four generators as VSGs, using the paper's parameters (virtual impedance j0.3 pu, H = 4 s, damping 5 pu on the 900 MVA VSG base). It's a Kron-reduced second-order swing model, $M\,d\omega/dt = P_{ref} - P_e(\delta) - D(\omega - 1) + u$ as in the paper's eq. (1), with a 0.5 ms zero-order-hold step and one B8-B9 circuit open after the fault. Without control it loses synchronism after the paper's fault, as in its Fig. 6. The controller follows the paper's parametrization: matrix networks $W(x)$, $Q(x)$, $S(x)$ and $R^{-1}(x)$ with $\pi(x) = -R^{-1}(x)S(x)^T x$ and $V(x) = x^T W(x) x$. It's trained with the paper's losses on 2,500 one-second trajectories (98,069 tuples), states relative to VSG1 and the region $X = \{|\Delta\delta| \le 2\pi, |\Delta\omega| \le 0.1 \text{ pu}\}$. I trained five seeds on the same data.

Every check uses 200,000 states sampled uniformly over X, plus 200,000 in each of two smaller boxes at 5% and 1% of X around the equilibrium (leaving out the inner 5% of each box), plus 512-start gradient attacks on each condition. I check the inequality as $F_D(x) = -x^T\Phi(x)x - (V(x^+) - V(x)) \ge 0$, which works because $s(x, \pi(x)) = -x^T\Phi x$ exactly.

Part 2. Why the training data can't see it

The inequality is trained on tuples where u is drawn uniformly over the input range. At those tuples the supply rate $s(x, u)$ is big, a median of about 85 against a storage change of about 0.006 per step, so the inequality is easy to satisfy. The theorem needs the closed loop, though, and at $u = \pi(x)$ the supply rate drops to its minimum over u, $-x^T\Phi(x)x$. That's the input with the least slack, and a uniform draw almost never lands near it. So a zero training loss says very little about the one case the proof actually uses.

Part 3. The scale mismatch

To first order in the step, $V(x^+) - V(x) = \Delta t \cdot \nabla V(x)^T f(x, \pi(x)) + O(\Delta t^2)$, so the decrease a step can deliver shrinks with Δt. The decrease the inequality asks for, $x^T\Phi(x)x$, doesn't depend on Δt at all. For the inequality to hold, Φ has to be about Δt times the continuous-time decrease rate, and the plant puts a rough ceiling on that rate. With $D/2M = 45/144 = 0.31$ s^{-1} on the system base, the swing energy decays on its own by about $2(D/2M)\Delta t \approx 3.1 \times 10^{-4}$ of itself per step. The controller can add to that, but with $|u| \le 0.3$ pu against $P_{ref} \approx 7$ pu it can't add orders of magnitude.

Near x* (box at 1% of X), median over states	Required decrease $x^T \Phi x$ / V	Delivered decrease (V − V^+) / V
Reproduced controller, five seeds	2.8% to 23.7% per step	0.0027% to 0.0071% per step
Restructured controller, five seeds	0.025% to 0.13% per step	0.23% to 0.27% per step
Passive budget, 2(D/2M)Δt	0.031% per step	

Table 1. The two sides of the inequality as fractions of V, 20,000 states per box. results/an/phi_budget_*.json.

So the reproduced controllers ask for 400 to 8,800 times more decrease per step than they deliver. The inequality isn't failing because the controller can't stabilize locally. It's failing because Φ, which sets the demand, was only asked to be positive definite and ended up two or three orders of magnitude too big for a 0.5 ms step on a plant with H = 4 s.

My first attempt at a fix shows why this has to be done in the structure and not with a penalty. When I just added the inequality at u = π(x) to the loss, its violation dropped to 0.01% of X within one epoch, but it got there by making Φ indefinite at every sampled state, which voids the theorem. Adding a $\Phi \succ 0$ penalty back didn't stop that within four epochs, because the two terms pull against each other.

Part 4. The structural change, five seeds

I kept the plant, region, data, original losses, network sizes, sampling and checks the same, warm-started each seed from its own reproduced controller, and changed two things. The Q head now outputs a positive definite matrix P(x), and $Q := S R^{-1} S^T - P$, so $\Phi = S R^{-1} S^T - Q = P(x) \succ 0$ holds by construction. And the inequality and the closed-loop decrease are trained at u = π(x), on states drawn half over X and half in boxes at 10%, 3% and 1% of X, for 26 epochs.

Seed	Controller	Inequality violated, X	Same, 1% box	$V(x^+) \le V(x)$ violated, X	Φ not PD, X / 1% box	Worst F_D found by attack
0	reproduced	84%	100%	47%	23% / 0%	−9.60
	restructured	8%	12%	5%	0% / 0%	−0.32
1	reproduced	79%	96%	47%	31% / 100%	−11.3
	restructured	23%	37%	14%	0% / 0%	−0.26
2	reproduced	99%	100%	48%	2% / 0%	−6.69
	restructured	14%	24%	8%	0% / 0%	−0.31
3	reproduced	81%	100%	48%	28% / 0%	−7.62
	restructured	11%	19%	7%	0% / 0%	−0.27
4	reproduced	75%	94%	48%	37% / 100%	−6.83
	restructured	14%	30%	8%	0% / 0%	−0.28

Table 2. Share of 200,000 sampled states violating each condition, all with the training margin ε set to 0 (see Limits). Attack is 512-start projected gradient descent over X. results/seeds/recheck_*.json and verify_*.json.

It helps on every seed, by 3 to 11 times on the inequality over X and 3 to 8 times near the equilibrium, and Φ is now positive definite everywhere instead of failing on up to 37% of X. The closed-loop decrease condition goes from 47% violated to 5% to 14%. One thing I noticed is that the reproduced controllers sit at almost exactly 50% on that condition near the equilibrium on every seed, which is what you'd expect if the one-step change in V is close to zero and its sign just depends on where the state is in its oscillation. After the change, the required decrease is 0.11 to 0.53 of what a step delivers, against 10 to 86 before, so the mismatch from Part 3 is gone.

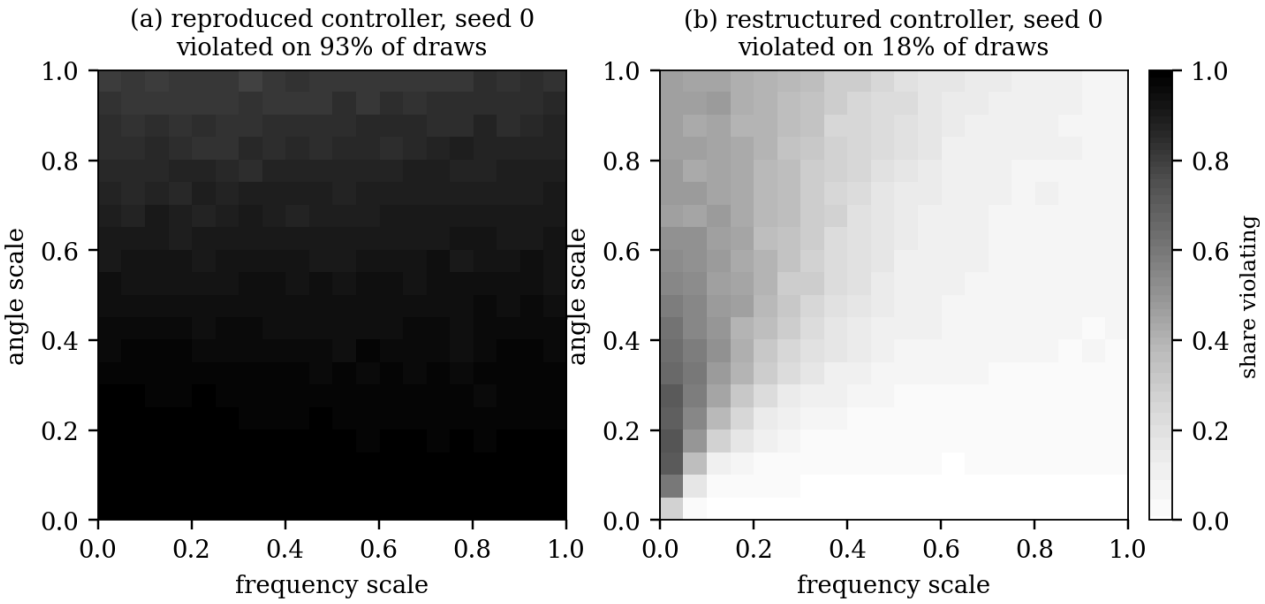


Figure 1. Share of draws violating the inequality, by how far out the state is in angle (vertical) and frequency (horizontal), each as a fraction of its half-width in X, 2,000 draws per cell. (a) Reproduced controller, violated nearly everywhere. (b) Restructured controller, where what's left sits in the left-hand band of low frequency deviation. Dense readout, not a certificate.

Part 5. Where what's left sits

What's left isn't spread over the region. On every seed it piles up at small frequency deviation and large angle (Figure 1b, Figure 2b). Among draws whose frequencies are within 20% of their half-width, 43% to 70% violate the inequality, against 5% to 14% when the frequencies are out past 80%. With every frequency deviation set exactly to zero, 47% to 76% of angle configurations violate it.

The swing model explains why that band is hard. At $\Delta\omega = 0$ the angles don't move to first order in one step, since $d\delta/dt = \omega_B \Delta\omega = 0$, so the one-step change of $V = x^T W x$ comes only through the frequency rows, $V(x^+) - V(x) \approx 2\Delta t\, (Wx)_\omega^T\, \dot{\omega}$, and with $x_\omega = 0$ that is $2\Delta t\, (W_{\omega\delta} x_\delta)^T\, \dot{\omega}$. The decrease therefore has to come entirely from the angle-frequency cross block of W, while $x^T \Phi x = x_\delta^T \Phi_{\delta\delta} x_\delta$ is still strictly positive. In the trained networks that cross block is small, $\|W_{\omega\delta} x_\delta\|$ is a median 0.013 to 0.026 of the frequency diagonal times $\|x_\delta\|$ after restructuring, and 0.09 to 0.11 before. So the fine-tune shrank the exact coupling this band needs.

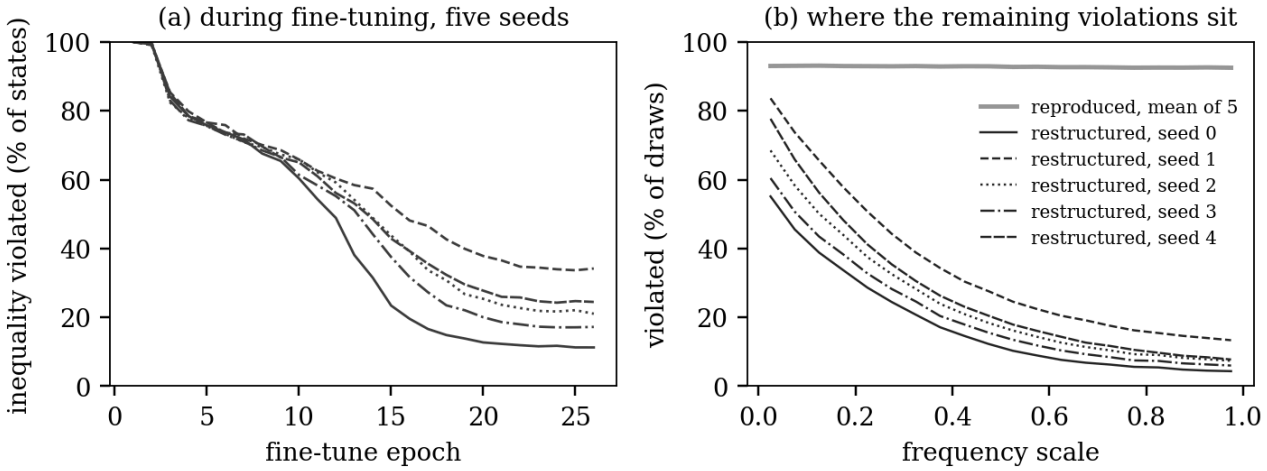


Figure 2. (a) Share of fresh states violating the inequality during fine-tuning, measured with the training normalization on the training mixture of region and boxes, so the endpoints are higher than Table 2. Seed 1 plateaus highest. (b) Violation rate against the frequency scale, averaged over angle scales, for the five restructured seeds and the mean of the five reproduced ones.

That leaves three explanations, and I can't tell them apart yet. The first is the parametrization. A Φ that is uniformly positive definite asks for a strict per-step decrease even where the plant can only start decreasing V a step later, so either $\Phi_{\delta\delta}$ has to vanish as $\Delta\omega \to 0$ or the condition has to be read over several steps. The second is training. Seed 1 plateaus at 34% on the training measure against 11% for seed 0 with the same budget, so some of what's left is optimization. The third is the region. X allows $|\Delta\omega|$ up to 0.1 pu, which is 6 Hz, while the trajectories that matter use a small fraction of that, so uniform sampling over X spends most of its states at frequency deviations the grid never reaches. That links to the fault result below.

Part 6. The fault, and what the input does

After the paper's fault (B8 fault at 0.2 s, cleared at 0.32 s by tripping one B8-B9 circuit) every one of the ten controllers leaves X at 1.87 s to 1.88 s, the same as with no control (Figure 3a). I first guessed that the input bound decided this case, so I tested it. I re-ran the whole pipeline with $|u| \le 2.7$ pu on the system base, which is 0.3 pu on the 900 MVA VSG base and matches how I converted the damping. The restructured controller there violates the inequality on 8.5% of X, about as well as the best seed above, and it still leaves X at 1.88 s. So the bound isn't what's stopping it. Before leaving X, the controllers use at most 0.007 to 0.026 pu of input with $|u| \le 0.3$, and 0.002 to 0.012 pu with $|u| \le 2.7$ (Figure 3b).

The reason is where the trajectory goes. From clearing until it leaves X, the largest frequency deviation averages 7% to 18% of the 0.1 pu half-width while the angles drift out to 2π. That is the low-frequency, large-angle band of Part 5, where the conditions are weakest and $\pi(x) = -R^{-1}S^{T}x$ is smallest, so the learned controller barely acts on the one trajectory the paper uses to show the enlarged region of attraction.

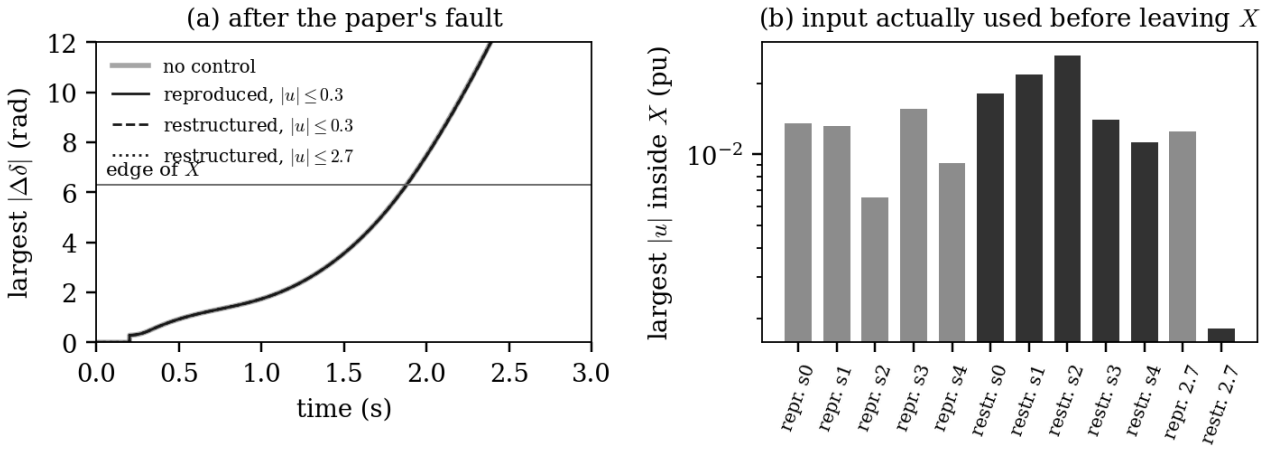


Figure 3. (a) Largest angle deviation after the paper's fault for no control and seed 0 at both input bounds. The curves lie on top of each other. (b) Largest |u| each controller applies before the trajectory leaves X, log scale. results/seeds/fault_seeds.json, results/u27/fault_u27.json.

How the cost grows

Step	Per seed	Wall time per seed
Reproduced controller, 60 epochs	98,069 tuples, batch 2,048	550 to 651 s
Restructure and fine-tune, 26 epochs	same tuples plus 2,048 closed-loop states per batch	361 to 438 s
Region check, dense plus attacks	600,000 states, 2,048 attack starts per condition	56 to 78 s

Table 3. Kaggle, 4 CPUs, seeds 1 to 4 and the |u| ≤ 2.7 run. Seed 0 ran on my laptop and took longer.

Where I'd like your read

Three things would change what I do next. Did you size Φ against Δt in any way, through how the states are normalized, a scaling in the supply rate, or the choice of X, or is Φ only constrained to be positive definite? How was X sampled in training, and is |Δω| ≤ 0.1 pu the region you meant the conditions to cover, or a filter on the data? And for the cost shaping in eq. (14) I used softplus(ε + s(x, π(x)) + l(x, π(x))) averaged over tuples, which gives a controller that barely acts on the fault trajectory, so I suspect I read that term differently from you. If one of these explains the gap, I'd like to rerun the five seeds your way and send you the comparison.

What this doesn't show

This is my rebuild, not your controller, and I made my own choices wherever the paper doesn't fix them (the epsilons, the cost-shaping form, mean-plus-max aggregation, an input-range penalty, |u| ≤ 0.3 pu), so your numbers could be different for any of those reasons. Every number here comes from dense sampling and gradient attacks, which find violations but can't prove there aren't any. I couldn't run a sound CROWN certificate because auto_LiRPA 0.7.2 doesn't support the GELU and softplus activations the networks use. I check the inequality with ε = 0, because a constant margin of 10^{-3} can't hold as x → 0 when both sides are around 10^{-6}. The input range |π(x)| ≤ 0.3 holds on every dense sample, but the attacks find corners of X where it's exceeded, up to 0.52 pu, on 2 of 5 reproduced and 4 of 5 restructured seeds. Current limiting isn't modeled. The code, the trained controllers and every file behind these tables are ready to share.